```
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------------------------------
# 1. SETUP ENVIRONMENT & FIELD WAYPOINTS
# ----------------------------------------------------
# Nodes layout: [0] Takeoff, [1 to 7] Crop Targets, [8] Landing Pad
waypoints = np.array([
    [10, 10],   # 0: Takeoff Point
    [20, 50],   # 1: Crop Target A
    [40, 80],   # 2: Crop Target B
    [60, 75],   # 3: Crop Target C
    [80, 50],   # 4: Crop Target D
    [70, 20],   # 5: Crop Target E
    [50, 10],   # 6: Crop Target F
    [30, 30],   # 7: Crop Target G
    [15, 12]    # 8: Landing Pad
])

num_nodes = len(waypoints)

# Precalculate Euclidean distance matrix between all points
dist_matrix = np.zeros((num_nodes, num_nodes))
for i in range(num_nodes):
    for j in range(num_nodes):
        dist_matrix[i, j] = np.linalg.norm(waypoints[i] - waypoints[j])

# ----------------------------------------------------
# 2. ACO ALGORITHM PARAMETERS
# ----------------------------------------------------
alpha = 1.0        # Importance factor for pheromones
beta = 2.0         # Importance factor for heuristics (closeness)
rho = 0.1          # Evaporation rate
Q = 100.0          # Pheromone deposit magnitude constant
num_ants = 15      # Population size of simulated drones per iteration
max_iterations = 60

# Initialize uniform pheromone levels across paths
pheromone = np.ones((num_nodes, num_nodes)) * 0.1

# Compute heuristic information (inverse of distance matrix)
with np.errstate(divide='ignore'):
    heuristic = 1.0 / dist_matrix
    np.fill_diagonal(heuristic, 0) # Avoid self-loop division issues

best_path = None
best_cost = float('inf')

# ----------------------------------------------------
# 3. CORE OPTIMIZATION LOOP
# ----------------------------------------------------
for iteration in range(max_iterations):
    all_paths = []
    all_costs = []

    for ant in range(num_ants):
        path = [0] # Drone always launches from Takeoff point (Node 0)
        unvisited = list(range(1, num_nodes - 1)) # Intermediate field targets

        current = 0
        while unvisited:
            # Evaluate probabilities for choosing adjacent available waypoints
            probs = []
            for next_node in unvisited:
                tau = pheromone[current, next_node] ** alpha
                eta = heuristic[current, next_node] ** beta
                probs.append(tau * eta)

            sum_probs = sum(probs)
            if sum_probs == 0:
                probs = [1.0 / len(unvisited)] * len(unvisited) # Baseline equal fallback
            else:
                probs = [p / sum_probs for p in probs]

            # Select path node via stochastic roulette-wheel choice
            next_node = np.random.choice(unvisited, p=probs)
            path.append(next_node)
            unvisited.remove(next_node)
            current = next_node

        path.append(num_nodes - 1) # Force termination at final Landing Pad node
```

```
        path.append(num_nodes - 1) # Force termination at final Landing Pad node
        all_paths.append(path)

        # Calculate total trajectory length
        cost = sum(dist_matrix[path[i], path[i+1]] for i in range(len(path) - 1))
        all_costs.append(cost)

        # Tracks historical global best path
        if cost < best_cost:
            best_cost = cost
            best_path = path.copy()

    # Apply pheromone evaporation
    pheromone *= (1.0 - rho)

    # Apply pheromone trail updates based on iteration performance
    for path, cost in zip(all_paths, all_costs):
        deposit = Q / cost
        for i in range(len(path) - 1):
            pheromone[path[i], path[i+1]] += deposit
            pheromone[path[i+1], path[i]] += deposit # Update bidirectionally

# ------------------------------------------------------
# 4. PLOT AND GRAPHICS RENDER
# ------------------------------------------------------
print(f"Optimal Flight Order: {best_path}")
print(f"Minimized Cost (Total Distance): {best_cost:.2f} meters")

plt.figure(figsize=(9, 7))
# Extract path sequence coordinates
path_coords = waypoints[best_path]

# Draw the optimized flight vector trajectory lines
plt.plot(path_coords[:, 0], path_coords[:, 1], 'g--', linewidth=2, label=f'Optimal Route ({best_cost:.2f}m)')
plt.quiver(path_coords[:-1, 0], path_coords[:-1, 1],
           path_coords[1:, 0] - path_coords[:-1, 0], path_coords[1:, 1] - path_coords[:-1, 1],
           scale_units='xy', angles='xy', scale=1, color='green', alpha=0.6)

# Highlight unique node types (Takeoff, Waypoints, Landing)
plt.scatter(waypoints[1:-1, 0], waypoints[1:-1, 1], color='blue', s=100, zorder=5, label='Crop Waypoints')
plt.scatter(waypoints[0, 0], waypoints[0, 1], color='red', marker='^', s=150, zorder=6, label='Takeoff Pad (Start)')
plt.scatter(waypoints[-1, 0], waypoints[-1, 1], color='black', marker='X', s=150, zorder=6, label='Landing Pad (End)')

# Label points individually for readability
for idx, pt in enumerate(waypoints):
    label = "Start" if idx == 0 else "End" if idx == num_nodes-1 else f"WP {idx}"
    plt.annotate(label, (pt[0]+1.5, pt[1]+1.5), fontsize=10, weight='bold')

plt.title('Ant Colony Optimization (ACO) — Agricultural UAV Flight Path Planning', fontsize=12, pad=15)
plt.xlabel('Field X-Coordinate (meters)')
plt.ylabel('Field Y-Coordinate (meters)')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper left')
plt.axis('equal')
plt.show()
```

```
Optimal Flight Order: [0, np.int64(7), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), 8]
Minimized Cost (Total Distance): 228.37 meters
```

